

Codeman Level 1: The Slow Script Challenge

1. Overview

In this level of the Codeman project, the player enters a classic Pac-man maze environment. However, a significant gameplay imbalance is introduced: the **Ghosts possess double the movement speed** of the player. To succeed, the player cannot rely on classic gameplay. They must act as developers to rebalance the game's mechanics. The objective is to write and inject a script that slows the ghosts, allowing the player to collect all dots and complete the level.



2. Dynamic Environment

- **Dynamic Obstacles:** Certain maze walls are programmed to move in specific directions, creating a fluid environment that helps the player to revise, correct, or optimize their scripts in a "safe zone" before re-entering the maze.



```
vector startPos;  
vector offset = <0.0, 4.0, 0.0>;  
float moveTime = 1.5;  
float stayTime = 2.0;  
  
default  
{  
    state_entry()  
    {  
        startPos = llGetPos();  
        llSetPrimitiveParams([PRIM_PHYSICS_SHAPE_TYPE, PRIM_PHYSICS_SHAPE_CONVEX]);  
    }  
  
    touch_start(integer num)  
    {  
        state moving;  
    }  
}  
  
state moving  
{  
    state_entry()  
    {  
        llSetKeyframedMotion([offset, ZERO_ROTATION, moveTime], []);  
  
        llSleep(stayTime);  
  
        llSetKeyframedMotion([-offset, ZERO_ROTATION, moveTime], []);  
  
        llSleep(moveTime);  
        state default;  
    }  
}
```

- **Camera Lock:** To maintain focus on the "coding challenge," the camera is locked in a top-down orthographic view. This prevents the player from losing sight of the maze, while forcing them to interact with the environment.

3. The Script Injection Workflow



The core mechanic of the level is the **Script Injection** process, managed via the **Codeman HUD**:

1. **Template-Based Coding:** Players are provided with a script template. They must implement logic within specific events—such as `touch_start` or `collision_start` to ensure high-speed interaction.

```

0 // =====
1 // Instructions for the User:
2 // 1. Open this script from your Inventory.
3 // 2. Write your logic inside the touch_start event.
4 // 3. Click "Save".
5 // 4. Rename this script to "SlowEffect" in your Inventory.
6 // 5. Drag and drop the "SlowEffect" script into your HUD.
7 // 6. Click the "UPDATE LOGIC" button on your HUD.
8 // =====
9
10 default
11 {
12     touch_start(integer total_number)
13     {
14         llOwnerSay("Power-up Activated: Sending SLOW signal to Ghost...");
15         llMessageLinked(LINK_THIS, 0, "SLOW_ON", NULL_KEY);
16     }
17 }

```

2. **Authentication & Security (PIN System):** Each Ghost object is protected by a unique **Remote Script Access PIN**. This ensures that only authorized scripts (e.g., the "Slow Script" designed for Level 1) can be remotely loaded into the Ghost's inventory.

```
integer GHOST_PIN = 12345;
```

```

state_entry()
{
    llListen(-100, "", NULL_KEY, "");
    llSetRemoteScriptAccessPin(GHOST_PIN);
    llSetRot(llEuler2Rot(<0,0,0>));
    UpdateGhostSprite();
    llSetTimerEvent(0.1);
}

```

3. **In-Game Execution:** Once injected, the user-written script communicates with the Ghost's primary AI controller through the `link_message` event.

4. The "Slow" Effect Management

Upon receiving the specific string message "SLOW_ON", the Ghost transitions into a **Vulnerable State**:

- **Speed Reduction:** The movement is drastically decreased according to a predefined float parameter.

```

float NORMAL_SPEED = 1.0;
float SLOW_SPEED = 0.2;

```

- **Visual Feedback:** The Ghost's primitive color changes to blue, providing the player with immediate visual confirmation that the script was successful.

```

link_message(integer sender_num, integer num, string str, key id)
{
    vector pos = llGetPos();

    if (str == "SLOW_ON")
    {
        speed = SLOW_SPEED;
        currentDirection = llVecNorm(currentDirection) * speed;

        slowDuration = 5.0;

        llSetColor(<0, 0, 1>, ALL_SIDES);
        llOwnerSay("Ghost effect: SLOWED");
    }
}

```

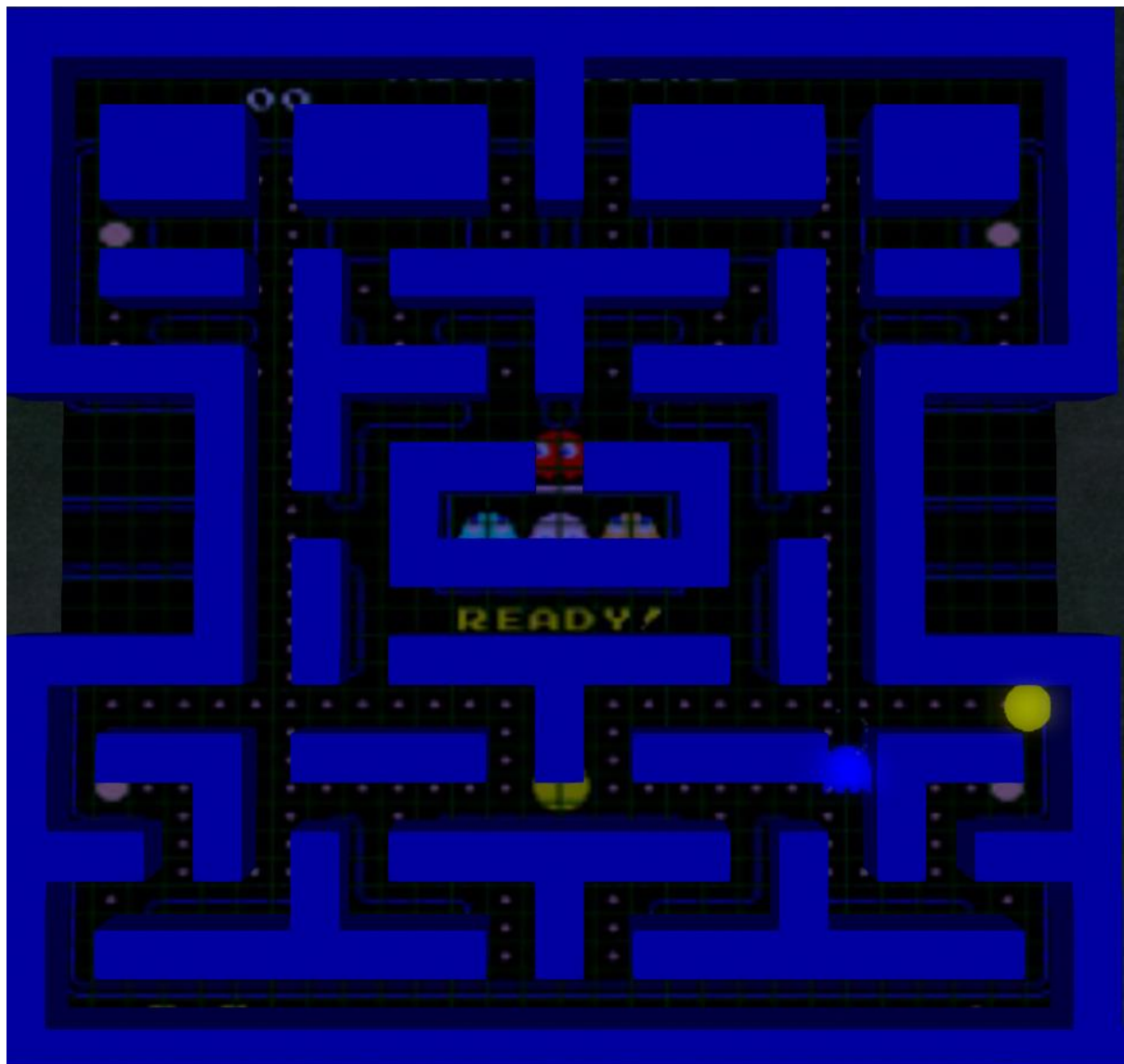
- **Autonomous Recovery:** The Ghost initiates an internal 5-second timer. Once expired, it automatically resets its speed and color to their original values without requiring external input.

```

timer()
{
    if(!canMove) return;

    if (slowDuration > 0)
    {
        slowDuration -= 0.1;
        if (slowDuration <= 0)
        {
            speed = NORMAL_SPEED;
            currentDirection = llVecNorm(currentDirection) * speed;
            llSetColor(<1, 1, 1>, ALL_SIDES);
            llOwnerSay("Ghost effect expired: Normal speed restored.");
            UpdateGhostSprite();
        }
    }
}

```



Ghost: Power-up Activated: Sending SLOW signal to Ghost...
Ghost: Ghost effect: SLOWED

Image 1: Ghost with slow effect enabled

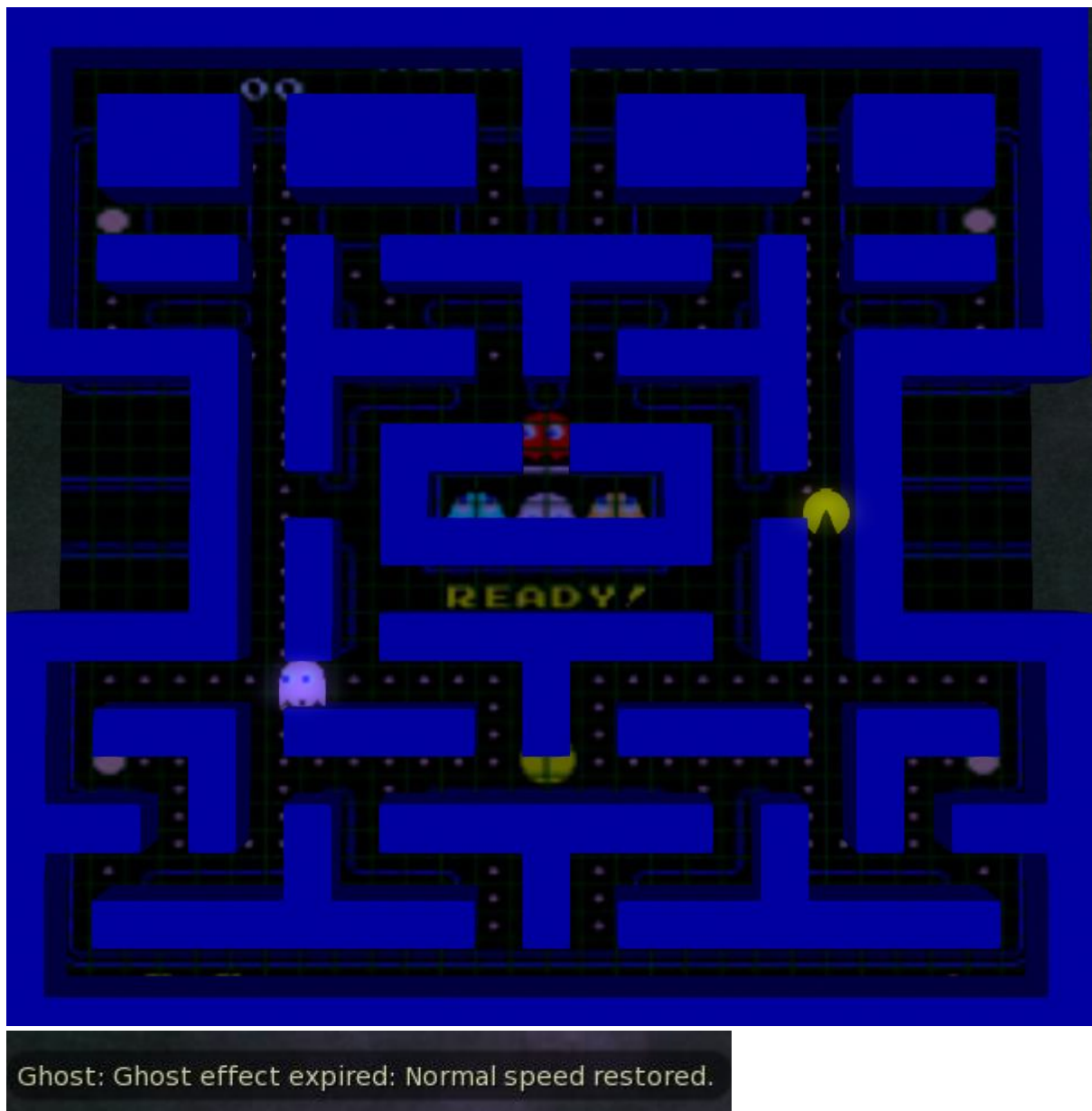


Image 2: Ghost after 5 seconds with slow effect disabled

5. Game Loop Fundamentals

To ensure a consistent and fair user experience, each level has two critical synchronization rules:

- **Initialization Trigger:** Ghosts remain in an "Idle State" (stationary) when the player first enters the maze. Their movement logic is activated only when the player physically touches the Pac-Man object, serving as a "Start Game" trigger.
- **Global Scene Reset:** When the player exits the maze portal, a global "STOP_GAME" signal is broadcast. All Ghosts immediately stop move, clear any

active effects, and teleport back to their **initial coordinates**, resetting the scene for the next attempt.